

ClimateReady Africa

Phase 3 finalisation and critical evaluation

Consolidated specification, architecture, verification and release documentation

Yannick Aurelien Wabo

Matriculation number: 9218017

Module: DLMCSPSE01

26 September 2026

Document control	Value
Document version	3.1 — Final documentation and prepared source
Reviewed software	Prepared source v1.0.1; rubric v1.0, dated 5 September 2026
Predecessor	Phase 2 submitted 17 September; available revised specification v2.1 dated 20 September
Release status	Prepared for 26 September submission; public GitHub and hosted verification still require student action
Scope decision	Public academic demonstration using fictional project information

Abstract

ClimateReady Africa connects a fictional mitigation-project profile and self-reported evidence to an explainable readiness score, critical gaps and a prioritised action plan. This consolidated specification records the project from approximately early August to the final submission deadline of 26 September 2026. It incorporates the assessor's planning, requirements and notation corrections. Version 1.0.1 closes the shared-write defect by rejecting public server mutations and retaining editable copies in each browser. Fifteen unit tests and four local HTTP checks passed on 25 September. Public GitHub publication, hosted verification and selected operational quality measurements remain outstanding. These limits are recorded explicitly so that implementation evidence is not mistaken for deployment or regulatory validation.

The separate two-page making-of abstract accompanies this report. This report is the maintained specification; the abstract is the concise Phase 3 submission narrative required by the university brief.

Table of contents

1 Feedback integration and evidence baseline	3
2 Problem, objectives and system scope	4
3 Methodology and complete project plan	5
3.1 Complete schedule and effort basis	6
4 Risk management	7
5 Requirements specification	8
5.1 Functional requirements for projects and evidence	8
5.2 Functional requirements for interpretation and output	9
5.3 Non functional requirements for integrity and reproducibility	10
5.4 Non functional requirements for operation and maintainability	11
6 System design and architecture	12
6.1 System scope and context	12
6.2 Building block view	13
6.3 Data model and request behaviour	14
7 Technology selection and design decisions	15
7.1 Central technology register	15
7.2 Architecture decisions and evolution	16
8 Implemented assessment model	17
9 Verification and test evidence	18
9.1 Executed unit and static checks	18
9.2 Workspace regression and local API checks	19
10 Requirements traceability and evaluation	20
11 Installation, operation and release	21
11.1 Local operation and reviewer walkthrough	21
11.2 GitHub, Cloudflare and final source freeze	22
12 Technical debt and critical reflection	23
13 Final submission assessment and packaging	24
14 List of abbreviations	25
15 Glossary	26
16 References and supporting evidence	27

1 Feedback integration and evidence baseline

The latest assessor comments are treated as change requests against Phase 2, not as new product features. Table 1 records the correction and the means of checking it. Earlier improvements—one canonical name, user stories, explicit risks and a central technology register—are retained (Wabo, 2026a; course assessor, n.d.).

Table 1. Response to the Phase 2 assessment

Assessor observation	Phase 3 correction	Location / check
Work packages remain vague.	A full lifecycle catalogue gives each package an ID, short description, requirement links and deliverable. A separate schedule records all phases, dependencies, dates, estimates and status.	§3; thirteen work packages and confirmed milestones
The contents page needs page numbers.	Continuous page fields and a populated, linked contents page identify actual section pages.	Contents; every page footer
NFR-06 mixes requirement and solution.	All NFRs state observable targets. NFR-06 removes technology and storage-format prescriptions; technical mechanisms are in design.	§5.4; §7.2
Notation and legends contradict UML; figure labels conflict.	Figures explicitly identify UML use-case, component and class diagram types. Components are straight rectangles; associations, dependencies and composition use their standard meanings.	Figures 1–3, §6
Report only tests actually performed.	Current unit, static and local API results are separated from unexecuted acceptance targets. The earlier shared-write defect and its regression checks remain traceable.	§9; evidence archive
Abbreviations and glossary are mixed.	Separate, independently alphabetised lists explain abbreviations and domain concepts.	§14 and §15
Heading sizes and spacing vary.	A single style hierarchy uses 14-point main headings, 12-point subheadings and 11-point body text with consistent spacing.	Whole document; rendered-page inspection

Evidence convention

“Verified” means a stated check was executed against the prepared v1.0.1 source on 25 September 2026. “Implemented” identifies behaviour present in code; “Open” identifies missing execution evidence or external release work. The student confirms submissions on 3 and 17 September and a final deadline of 26 September. Work-package dates before those milestones are reconstructed planning windows, not an invented daily activity log. Actual hours were not recorded in the information supplied.

The v1.0.0 export was the code input; v1.0.1 is delivered with file hashes. No public repository URL, final commit or production URL has been supplied. The Phase 2 design baseline reviewed was the 20 September revision. A document uploaded on 17 September is also included for comparison; the student must confirm the exact submitted file.

2 Problem, objectives and system scope

The project addresses a preparation problem: a developer needs to connect a mitigation activity's profile with the evidence and responsibilities relevant to its intended carbon-market pathway. Fragmented working documents can make this relationship difficult to review. The application offers a structured screening workflow; the claim that it reduces preparation time remains a hypothesis because no comparative user study has been conducted.

The engineering objective is to implement a reproducible, understandable assessment and make it accessible as a public academic demonstration. The primary audience is project developers and consultants. Public institutions and technical partners are secondary readers of assessment results; the software does not take over their decision-making responsibilities.

Table 2. Controlled product scope

Within the final demonstration	Outside the agreed release
Portfolio and four labelled fictional projects; create/edit a fictional profile; save and reopen an assessment.	Real or confidential project information; organisational accounts and production tenancy.
Five pillars; up to fourteen applicable criteria; evidence notes; score, completeness, gaps and actions.	Automatic legal eligibility determination; document authenticity checks; carbon-credit issuance or registry transfers.
Framework explanation; structured text report; repeatable installation; cloud demonstration.	Payment, live monitoring ingestion, country-rule completeness, rubric administration and application-generated PDF reports.
Read-only shared examples and a resettable visitor sandbox are required for public release.	Anonymous modification of shared examples is not accepted as a completed public-demo design.

Success criteria

The core workflow must satisfy FR-01–FR-10; the score must be reproducible from applicable weights and ratings; unresolved critical evidence must block the highest readiness status. The final demonstration must prevent cross-visitor changes, provide a working public URL and reproduce from the delivered source. The final GitHub release and implementation ZIP must contain identical source files, as required by the university (IU International University of Applied Sciences [IU], n.d., pp. 4–5).

The rubric is an academic product model. It draws on named domain references but its weights and thresholds are design choices. Neither a high score nor the label “Evidence ready” establishes host-Party approval, authorisation, registration or independent verification. Official rules remain external reference material (UNFCCC, n.d.).

3 Methodology and complete project plan

The project follows a solo iterative Agile approach: clarify an outcome, implement a small increment, check its behaviour and reconcile the specification with evidence. Yannick Aurelien Wabo owns all work packages; the assessor reviews the academic submissions. Reading and coding began approximately one month before the first submission. The plan therefore uses 3 August as an approximate planning origin, not an asserted exact start date. Requirements and implementation overlap intentionally.

Table 3a. Complete work package catalogue across all phases

ID	Brief work package and output	Requirement or artifact reference
WP-01	Read course and domain material; define problem, audience and boundaries. Output: project profile.	Scope; reference register
WP-02	Specify stories, quality targets, risks and schedule. Output: requirements and initial plan.	FR-01–10; NFR-01–10; R-01–08
WP-03	Select architecture and technologies. Output: context, building blocks and decisions.	§§6–7; ADR-01–07
WP-04	Build initial interface and rule prototype. Output: early runnable workflow.	FR-01; FR-06–08
WP-05	Consolidate and submit Phase 1. Output: submitted conception document.	Milestone M1; §§2–7
WP-06	Develop profiles, validation and persistence. Output: editable project workflow.	FR-01–05; NFR-01–02
WP-07	Complete scoring, actions and export. Output: end-to-end assessment.	FR-06–10; NFR-03
WP-08	Test and refine code and specification; submit Phase 2. Output: development submission.	Milestone M2; §9
WP-09	Review assessor feedback and release risks. Output: correction and defect register.	Table 1; R-04–06; §12
WP-10	Isolate visitor copies and add regression checks. Output: prepared source v1.0.1.	FR-03–05; NFR-02/04; UT-08–15; API-01–04
WP-11	Finalise report, UML views and abstract. Output: paginated final documentation.	All specification sections; two-page abstract
WP-12	Publish GitHub source, deploy and check operation. Output: verified public links.	NFR-05/07/08/10; deployment evidence
WP-13	Match source archives, complete formalities and submit. Output: final frozen package and receipt.	Milestone M3; §13

3.1 Complete schedule and effort basis

Historical package windows are retrospective approximations within the confirmed phase boundaries. Effort figures are reconstructed planning estimates, not actual timesheets. The final three dates are current targets. Dependencies indicate required inputs; iterative packages can overlap. C = completed at phase level as reported or evidenced; V = implementation checked locally; O = external action or acceptance still open.

Table 3b. Full lifecycle schedule as of 25 September 2026

WP	Window in 2026	Depends on	Estimate	Status and exit evidence
01	Approx. 3–12 Aug	—	10 h	C; project profile and references
02	Approx. 8–20 Aug	01	12 h	C; Phase 1 requirements and risks
03	Approx. 15–26 Aug	02	10 h	C; initial design; subsequently revised
04	Approx. 20 Aug–3 Sep	02, 03	20 h	C; initial code work reported by student
05	Approx. 27 Aug–3 Sep	01–04	8 h	C; M1 submitted 3 Sep, confirmed
06	Approx. 4–10 Sep	04, 05	16 h	C; exported profile and persistence code
07	Approx. 8–15 Sep	04, 06	18 h	C; domain functions and text export
08	Approx. 11–17 Sep	06, 07	12 h	C; M2 submitted 17 Sep, confirmed
09	Approx. 18–24 Sep	08	6 h	C; assessor feedback and shared-write finding
10	25 Sep	09	6 h	V; 15 unit and 4 local HTTP checks
11	25–26 Sep	09, 10	5 h	Prepared; final student review required
12	25–26 Sep	10	4 h	O; GitHub, Cloudflare and hosted checks
13	26 Sep	11, 12	3 h	O; source equivalence, URLs and receipt

The reconstructed estimate is 130 person-hours: 60 for Phase 1, 46 for Phase 2 and 24 for Phase 3. These totals must not be interpreted as measured effort. Milestones M1 (3 September) and M2 (17 September) are actual submissions; M3 (26 September) is the confirmed deadline. The exact platform cut-off time remains to be checked by the student. No mandatory pre-submission work is scheduled after the deadline.

The remaining route is publication and operation, final evidence reconciliation, source equivalence and submission. On 25–26 September, an unresolved deployment issue lasting two hours triggers immediate replanning and, if necessary, contact with the assessor before the cut-off. Optional features are deferred. Open mandatory deliverables are not relabelled technical debt to imply course compliance.

4 Risk management

Yannick Wabo owns and reviews all risks at each backlog review and before the release freeze. Probability (P) and impact (I) use a 1–5 ordinal scale; $P \times I$ is a prioritisation aid, not a statistical probability. A realised defect is also entered in the technical-debt register, rather than being hidden within a future risk.

Table 4. Risks, triggers and specific responses

Risk / $P \times I$	Trigger and prevention	Contingency / evidence
R-01 Scope growth $4 \times 4 = 16$	A new item exceeds four hours. Keep FR-01–FR-10 fixed; estimate before acceptance and exchange optional scope of similar effort.	Defer the lowest-priority optional item; record the scope and schedule change.
R-02 Invalid screening logic $3 \times 5 = 15$	An expected score or critical gate fails. Publish weights and applicability; keep deterministic regression cases.	Correct the rule, increase rubric version and rerun affected and regression cases before restoring the result.
R-03 Guidance changes $3 \times 4 = 12$	A referenced rule or national process changes. Review primary sources before release and date the rubric.	Record the affected criterion and decision; revise or qualify it and flag comparisons across rubric versions.
R-04 Schedule slippage $4 \times 4 = 16$	A mandatory task remains blocked for two hours on 25–26 September. Review progress after each release task and protect time for upload and verification.	Defer optional polish; resolve the blocker or contact the assessor before the deadline. Never invent a completed deployment.
R-05 Shared data alteration $4 \times 5 = 20$	The v1.0.0 review exposed shared writes. Version 1.0.1 rejects server mutations and stores visitor copies separately; rerun regressions after edits.	Do not publish the writable baseline. Release only after two-session isolation and direct-request checks pass.
R-06 Runtime divergence $3 \times 4 = 12$	Local build passes but Docker or hosted execution fails. Use locked dependencies and rehearse migration plus startup.	Return to the previous working release; repair configuration and repeat the failed environment check.
R-07 Credential exposure $2 \times 5 = 10$	A deployment token appears in tracked content. Keep tokens in deployment secrets; review source and history before public push.	Revoke and rotate the token, remove exposure and verify the replacement deployment. Identifiers alone are not passwords.
R-08 Misinterpretation $3 \times 5 = 15$	A reader treats screening as approval. Label fictional data and explain the result beside its score and export.	Revise misleading copy and examples; withhold any unsupported claim of eligibility or certification.

Risk priority is a release decision, not a promise that all uncertainty will be removed. The shared-write code defect is fixed and locally checked; hosted confirmation remains open. Limited external validation of the rubric remains an explicitly accepted academic limitation.

5 Requirements specification

5.1 Functional requirements for projects and evidence

All ten functional requirements are Must requirements for the bounded demonstration. Each uses a user story and a testable outcome. Implementation mechanisms and acceptance evidence are separated into §§6–10. The public-demo revision changes where visitor work may be stored; it does not permit visitors to overwrite shared examples.

FR-01 — View and open projects

As a visitor, I want to select a fictional example or my own demonstration project so that I can begin an assessment.

Acceptance: Given an initial workspace, when it opens, then four labelled fictional examples are available; selecting one shows its profile and assessment.

FR-02 — Create a fictional project

As a visitor, I want to enter a project profile so that the assessment reflects the intended country, sector, pathway and use.

Acceptance: Given valid details, when saved, then a distinct project appears. Invalid values are rejected with a useful message; no partial record is created.

FR-03 — Change assessment scope

As a visitor, I want to edit the profile so that the assessment remains aligned with the project design.

Acceptance: Given saved responses, when country, sector, pathway or intended use changes, then earlier responses are cleared. A stale update must not overwrite a newer saved version.

FR-04 — Complete and resume an assessment

As a visitor, I want to rate applicable criteria and retain evidence notes so that I can resume my demonstration later.

Acceptance: Given an assessment, when saved and reopened in the same supported workspace, then its ratings and notes are restored. Shared examples remain unchanged by visitor experiments.

FR-05 — Support documented ratings

As a visitor, I want a supporting note to be required for a documented rating so that full credit has an inspectable basis.

Acceptance: Given Documented with fewer than ten characters after trimming leading and trailing whitespace, when saved, then the affected criterion is identified and the save is rejected.

Validation of note length establishes a minimal reference field, not evidence authenticity. This interpretation corrects the earlier phrase “ten non-space characters”, which did not precisely describe the implemented trim-and-length rule.

5.2 Functional requirements for interpretation and output

FR-06 — Apply pathway and sector scope

As a visitor, I want irrelevant criteria excluded so that the result corresponds to the selected project scope.

Acceptance: Given PACM contribution use, authorisation and accounting criteria are excluded; given Article 6.2 or authorised use, they apply. Reversal risk applies to forestry and land use, biochar and Other.

FR-07 — Understand the result

As a visitor, I want a readiness score, completeness value and status so that I can distinguish assessment progress from evidence maturity.

Acceptance: Given ratings, the score is an integer from 0 to 100. Evidence ready requires score ≥ 80 , every applicable item rated and every critical item documented with a sufficient note.

FR-08 — Prioritise next actions

As a visitor, I want unresolved criteria translated into actions so that I can sequence further preparation.

Acceptance: Given unresolved items, critical items appear first and higher weights break ties. Each action identifies the criterion, next step and responsible role.

FR-09 — Export a point-in-time report

As a visitor, I want to download the assessment so that I can review its result and evidence away from the application.

Acceptance: Given an assessment, the export includes profile, scope, score, completeness, applicable criteria, notes and actions, plus rubric version, time and limitations. The downloaded file shall be readable and identify its assessment scope; the chosen text format is specified in §7.

FR-10 — Inspect the framework

As a visitor, I want to understand the rubric and its limits so that I can interpret the result appropriately.

Acceptance: The framework explains five pillars, rating values, formula, status conditions, reference date and sources. All public entry and export paths identify the academic, fictional-data scope.

A downloadable application PDF and an automatic pathway recommendation were proposed earlier but are not implemented in the reviewed release. The user selects the pathway; the application adapts the screening criteria. The academic report and required abstract are separate PDF deliverables and do not imply application-level PDF export.

5.3 Non functional requirements for integrity and reproducibility

Non-functional requirements specify observable quality outcomes. Technologies and commands are justified separately in §7. Thresholds below are project acceptance targets, not findings that have already been achieved. This distinction follows the general requirements-engineering concern for clear and verifiable specifications; no full conformity with ISO/IEC/IEEE 29148 is asserted (ISO, 2018).

Table 5. Quality requirements NFR-01–NFR-05

ID / quality	Required outcome	Acceptance observation
NFR-01 Input integrity	Reject names outside 3–120 characters, descriptions over 3,000, evidence over 5,000, more than 14 answers, invalid enumerated choices and mitigation estimates outside integer 0–1,000,000,000.	Boundary values accepted or rejected as specified; rejected requests leave the saved record unchanged.
NFR-02 Update integrity	For two edits originating from one saved version, zero stale saves shall silently overwrite the newer accepted version.	Attempt the second stale edit; report the conflict and retain the accepted data.
NFR-03 Explainability	For 100% of displayed results, the reader shall be able to recover applicable weights, rating values, score calculation, completeness, critical status conditions and unresolved actions.	Recalculate a supplied example from its displayed or exported assessment and obtain the same integer score and status.
NFR-04 Visitor isolation	Zero anonymous mutation attempts shall change shared examples or a different visitor workspace. Reset shall affect exactly the selected workspace.	Use two independent browser contexts and direct mutation requests; confirm that shared and other-visitor data remain unchanged.
NFR-05 Reproducibility	A technically competent reviewer shall be able to start the submitted application from a clean supported environment within 30 minutes, excluding account creation and network download delays.	Record setup on the documented local environment and a clean container environment; verify the core workflow and retain the commands and outcome.

NFR-04 is mandatory for the public demonstration. API-02–04 confirm rejection of shared writes; workspace unit tests verify separation of independent stores. A check with two real browser profiles in the hosted environment remains outstanding.

The observed local execution supports NFR-02 and parts of NFR-01, NFR-04 and NFR-05. It does not establish all input boundaries, clean-machine setup time or repeatability in Docker and production. The evidence mapping in §10 preserves these distinctions.

5.4 Non functional requirements for operation and maintainability

Table 6. Quality requirements NFR-06–NFR-10

ID / quality	Required outcome	Acceptance observation
NFR-06 Maintainability	A maintainer using the documentation shall complete one criterion-weight or recommended-action change and verify affected and unaffected results within 60 minutes. Previously saved project information shall remain intact.	Perform and time the maintenance scenario on a temporary branch; retain the change, expected results and regression output.
NFR-07 Accessibility	Portfolio, profile, assessment and export tasks shall be keyboard operable, use labelled controls and visible focus, and retain essential content at 200% zoom and 360 px viewport width.	Complete the four tasks with keyboard only and inspect the specified zoom and width; record any obstruction. This is selected-criteria evaluation, not a full WCAG conformance claim.
NFR-08 Responsiveness	In the hosted demonstration, the 95th percentile of project-load and assessment-save completion time shall be below 2 seconds with ten projects and ten concurrent visitor sessions over five minutes.	Record elapsed user-action time and environment. Measure both project opening and successful save confirmation, with ten projects in each session; record browser version, device, network and test location.
NFR-09 Data minimisation	The demonstration shall request no passwords, email addresses, identity documents or payment details; shipped project information shall be fictional.	Inspect every data-entry field, stored field and seed record; confirm that no required identity or payment data is collected.
NFR-10 Recoverability	After a failed release, a maintainer shall be able to restore the previous working demonstration and its approved fictional baseline within 60 minutes using the recorded release materials.	Rehearse rollback and record the source version, data recovery action, elapsed time and restored core workflow.

NFR-06 now describes the desired maintenance capability and a measurable scenario. The pure domain module, typed boundaries and combined quality command are selected means of achieving that outcome, described in §7.2. Passing the command alone does not demonstrate the 60-minute requirement.

The accessibility targets are informed by selected WCAG 2.2 provisions on keyboard access, focus, labels, text resizing and reflow (W3C, 2024). Browser completion, timing and rollback observations remain release work; they are not added to the list of tests already performed.

6 System design and architecture

6.1 System scope and context

The architecture describes prepared version 1.0.1. Figure 1 uses a UML use-case diagram to define the functional system boundary and the visitor's interactions. Figure 2 supplies the technical dependencies. The maintainer changes source, reference material and deployment configuration outside the visitor interface; the application has no administrator screen. Official reference sites are consulted manually and are not executing integrations.

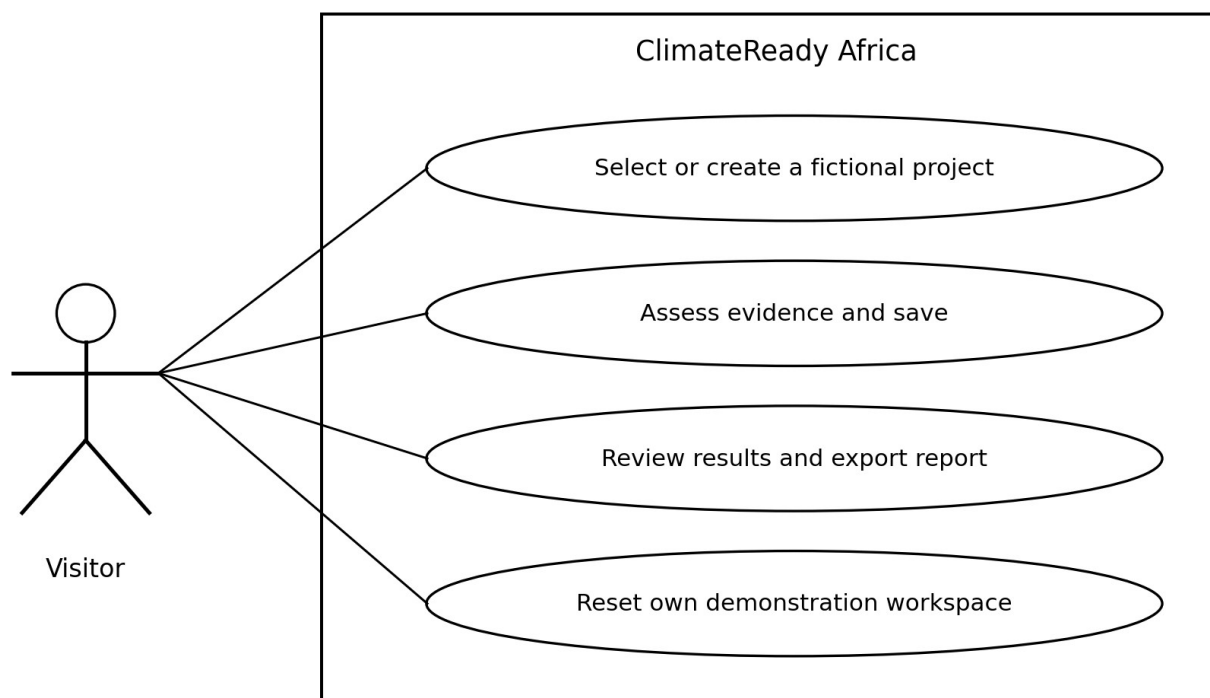


Figure 1. UML use-case diagram of system scope and visitor context

The rectangle is the system subject, ellipses are use cases and the stick figure is an external actor. Solid undirected lines are actor–use-case associations; they do not represent data flow. Notation follows UML 2.5.1 (OMG, 2017).

The visitor selects or creates a fictional project, records evidence, reviews results, exports a report and resets their own workspace. The application serves four shared templates, but visitors edit local copies. Different browser profiles have separate workspaces; people sharing one profile share its local data. Clearing browser storage removes saved work. These consequences are disclosed in the interface.

The software boundary comprises the browser interface and delivered Worker code. Cloudflare D1 is an external managed service supplying fictional templates. Browser storage holds visitor work; there is no cross-device synchronisation or authenticated tenancy. These are explicit design choices under NFR-04 and NFR-09, rather than technology prescriptions within the requirements.

6.2 Building block view

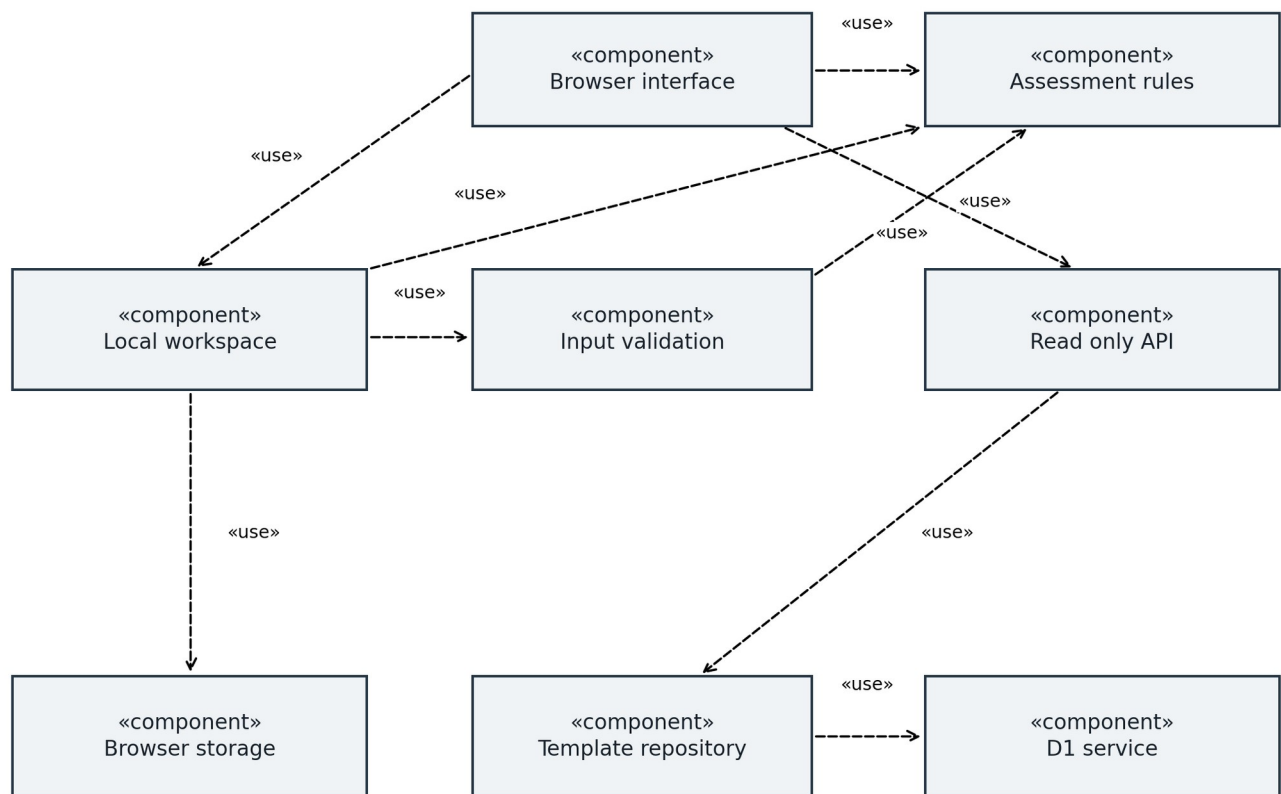


Figure 2. UML component diagram of implementation dependencies

Straight rectangles marked «component» represent components. Dashed open-headed «use» arrows are UML Usage dependencies, directed from client to supplier. They express static dependency, not request/response timing or data movement (OMG, 2017).

Table 7. Component responsibilities and source locations

Component	Source location	Responsibility
Browser interface	app/page.tsx	Portfolio, profile, evidence, results, export and reset.
Local workspace	lib/local-workspace.ts	Validated saves, revision checks, local copies and reset.
Assessment rules	lib/assessment.ts	Applicability, weights, score, status, actions and report.
Input validation	lib/validation.ts	Profile and update constraints reused by the workspace.
Read only API	app/api/projects/	List templates; reject public POST and PUT with 403.
Template repository	db/projects.ts	Seed fictional examples and select only demo rows from D1.

Browser storage is accessed through the workspace module; D1 is accessed by the server repository. Web Locks serialise local writes and revision checks reject stale edits. The Worker/Vinext adapter serves the interface and routes. The diagram shows dependencies rather than deployment containers.

6.3 Data model and request behaviour

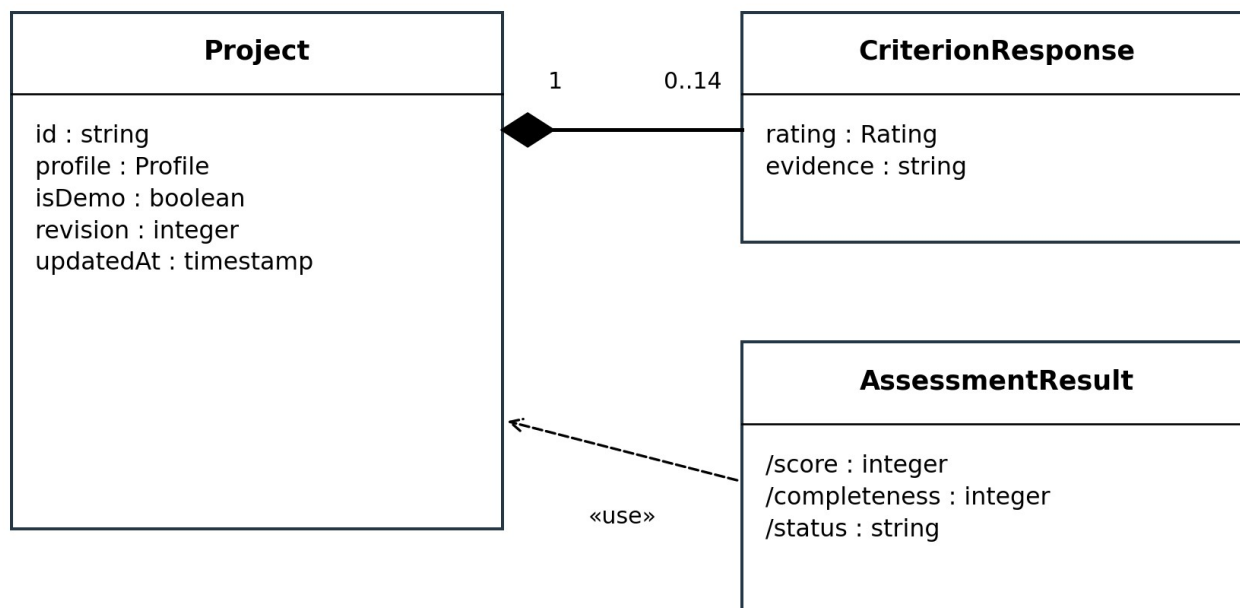


Figure 3. UML class diagram of the logical assessment aggregate

A class rectangle separates its name from attributes. The filled diamond denotes composition at Project, with 0..14 owned response entries per project. The dashed Usage dependency means AssessmentResult uses Project. Slash-prefixed attributes are derived (OMG, 2017).

The logical aggregate is represented twice: D1 holds fictional templates and each browser holds a versioned workspace envelope containing editable copies and created projects. A project records its profile, responses, example flag, revision and timestamps. Scores, completeness, status and actions are calculated. There is no separate assessment-history table; rubric migration remains documented debt. The class diagram is not a claim that each class is a database table.

Table 8. Server API and local workspace operations

Operation	Result	Failure or integrity behaviour
GET /api/projects	200 with fictional templates only; initialise missing templates.	503 when D1 cannot be read.
POST /api/projects	No server creation permitted.	403 for every request.
PUT /api/projects/:id	No server mutation permitted.	403 for every request, including direct anonymous calls.
Local create or save	Validate; persist; then update interface.	Reject invalid evidence, stale revision or failed storage write.
Local reset	Confirm; replace only current workspace with template copies.	New revisions invalidate stale open edits.

The workspace permits at most 100 projects. A current browser with Web Locks on HTTPS or localhost is required for saving. Revisions and locking support NFR-02; they are not an authentication mechanism. Browser storage can be changed by its owner, so its contents must not be treated as independently verified evidence.

7 Technology selection and design decisions

7.1 Central technology register

Table 9 centralises technology choices and reasons. Versions come from the submitted lockfile; links identify primary documentation. Browser storage choices are included here rather than in the NFRs.

Table 9. Selected technologies, reasons and trade-offs

Choice / baseline	Role and reason	Trade-off / primary source
React 19.2.6; TypeScript 5.9.3	Component-based interface and shared types reduce contract drift across client and server.	State complexity remains; types do not validate incoming JSON. react.dev ; typescriptlang.org/docs
Next 16.2.6 API surface; Vnext 0.0.50; Vite 8.0.13; Cloudflare plugin 1.37.1	Next-compatible routes and a Worker adapter provide one TypeScript deployment and integrated builds.	Young adapter creates compatibility risk. Pin versions and check deployment. github.com/cloudflare/vnext ; vite.dev/guide
Workers; Wrangler 4.92.0	One cloud execution environment and a versioned deployment CLI fit the solo project.	Cloudflare account and configuration required. developers.cloudflare.com/workers
Cloudflare D1	Managed SQLite-compatible storage supplies shared fictional templates through a server-side binding.	Platform coupling; future history and tenancy require redesign. developers.cloudflare.com/d1
Drizzle ORM 0.45.2; kit 0.31.10	Typed schema and explicit migration generation make the data model inspectable.	Repository operations use parameterized D1 SQL directly; not all access is ORM-generated. orm.drizzle.team/docs/overview
Zod 3.25.76 baseline	Runtime schema validation constrains untrusted payloads.	Validation must be tested at boundaries; major upgrades need review. zod.dev
Tailwind CSS 4.2.1; UI primitives	Shared styling and form/dialog primitives support a consistent interface.	Component availability is not proof of accessibility. tailwindcss.com/docs ; package.json inventory
Node ≥22.13; ESLint 9.39.4; TypeScript compiler	Built-in tests, lint and type checks support repeatable verification without another test framework.	Local review: Node 24.19; CI: Node 22. Static checks do not verify UI or cloud. nodejs.org/api/test.html ; eslint.org/docs/latest
Git; GitHub Actions	Version history, repeatable quality jobs and release traceability.	A workflow file is not evidence of a successful hosted run. docs.github.com/actions
Docker and Docker Compose	A documented local environment addresses the required container packaging.	Uses a development Worker/D1 emulator, not the production cloud service. docs.docker.com/compose
Browser localStorage and Web Locks	Keep fictional visitor work in the current browser; serialize writes across tabs.	No cross-device sync; clearing data loses work. developer.mozilla.org/en-US/docs/Web/API/Web_Locks_API

7.2 Architecture decisions and evolution

Table 10. Architecture decision records

ID / decision	Rationale and consequence	State
ADR-01 Deterministic rubric	A reader must reproduce a score. Pure weighted rules support inspection and tests; usefulness still depends on the judgement encoded in the rubric.	Implemented
ADR-02 One TypeScript deployment	Shared types and one Worker reduce coordination for a solo project. This trades operational simplicity for Cloudflare/Vinext coupling.	Implemented
ADR-03 Project aggregate	D1 supplies templates; a versioned local workspace holds visitor copies. The latest profile and responses are stored together; history is deferred.	Implemented
ADR-04 Code-versioned rubric	Criteria and actions are reviewed with source changes. The domain module and quality command support maintainability; the timed NFR-06 scenario remains unverified.	Implemented; version persistence debt
ADR-05 Visitor isolation	Server mutations are rejected and edits remain in browser storage. Web Locks and revisions serialize concurrent edits; reset is local. This meets the fictional-data boundary without an account system.	Implemented; local regression evidence
ADR-06 Optimistic concurrency	Use Web Locks and stored revisions to serialize saves and reject stale writes. UT-11 and UT-13 confirm revision checks; real browser concurrency remains to be observed.	Implemented and locally checked
ADR-07 Structured text export	A readable, deterministic text report provides the required assessment content at low implementation cost. Formatted application PDF is deferred.	Implemented

The original FastAPI/PostgreSQL/JWT proposal was replaced by the inspected TypeScript Worker/D1 implementation. Accounts and an administration interface are outside the agreed fictional-data release. The reporting outcome is now a text export, and pathway selection is explicit rather than an automated recommendation. These changes reduce scope and operational work; they must be stated because the Phase 1 proposal is retained in the submission history.

The application is not platform-independent merely because it includes Docker. The supplied container runs local development infrastructure, while production uses Worker bindings and D1. This limitation is accepted only with functioning installation and cloud instructions, both of which still require a clean release rehearsal.

8 Implemented assessment model

The maximum rubric contains fourteen criteria across five pillars. A criterion contributes only if applicable to the selected profile. Its value is 0 for Not yet or unanswered, 0.5 for In progress, and 1 for Documented with at least ten characters after trimming the evidence note. A documented rating without that note contributes 0. These are product rules, not official UNFCCC ratings.

Score = $\text{round}[100 \times \Sigma(\text{applicable weight} \times \text{rating value}) / \Sigma(\text{applicable weight})]$. Completeness = $\text{round}[100 \times \text{rated applicable criteria} / \text{all applicable criteria}]$.

Table 11. Rubric v1.0 weights and critical criteria

Pillar	Criteria and base weights	Maximum pillar weight
Policy and eligibility	NDC alignment 8; host-country participation 5 (conditional).	13
Carbon integrity	Additionality 12*; methodology and baseline 12*; reversal risk 6 (conditional); avoidance of double counting 8*.	38
MRV and verification	Monitoring and reporting 10; independent validation and verification 7.	17
People and safeguards	Environmental and social safeguards 7*; stakeholder participation 6; rights and benefit sharing 5*.	18
Market pathway	Project approval 5*; international-use authorisation 5* (conditional); corresponding adjustments and reporting 4* (conditional).	14

* Critical criterion. Base weights total 100 before exclusions. For a Senegal solar PACM contribution example, reversal risk, international-use authorisation and accounting are excluded, leaving weight 85. If only NDC alignment is documented, the score is $\text{round}(100 \times 8/85) = 9$; completeness is $\text{round}(100 \times 1/11) = 9$. This worked example demonstrates the arithmetic rather than empirical project quality.

Status rules

No rated items yields Not assessed. Evidence ready requires a score of at least 80, complete rating coverage and no unresolved critical criterion. Otherwise, scores of at least 50 yield Needs work and lower scores yield Early stage. Actions list unresolved critical items first, then order by weight.

Validity and interpretation

The model measures declared preparation evidence, not mitigation impact or probability of official acceptance. Its weights have not been statistically calibrated, and ten characters cannot establish document quality. Four fictional examples demonstrate software behaviour but do not establish representativeness across African jurisdictions. A score comparison is meaningful only under a compatible rubric and project scope.

9 Verification and test evidence

9.1 Executed unit and static checks

On 25 September 2026, prepared v1.0.1 was checked locally on Linux with Node 24.19.0. Lint, 15 Node unit tests and TypeScript checking passed. The build command exited successfully; its generated client and Worker outputs were also subjected to a deployment dry run. Exact logs accompany the source package. No GitHub Actions execution or production deployment is inferred from these local results.

The review reused the dependency tree from the earlier locked installation. A fresh offline npm ci attempt could not retrieve one uncached package. Therefore the review does not establish a successful clean-machine installation; the supplied lockfile and CI workflow provide the procedure for that remaining check.

Table 12a. Existing unit cases retained in v1.0.1

ID	Concrete expectation	Result
UT-01	Base weights total 100; criterion applicability follows pathway and sector.	Pass
UT-02	Unresolved critical evidence prevents Evidence ready despite a high score.	Pass
UT-03	Completeness is independent of readiness score.	Pass
UT-04	Text report contains project, actions and screening limitations.	Pass
UT-05	A valid profile is accepted.	Pass
UT-06	Article 6.2 with contribution use is rejected.	Pass
UT-07	Unknown country and negative mitigation value are rejected.	Pass

The cases are in tests/assessment.test.mjs and tests/validation.test.mjs. UT identifiers are report references; one case can contain more than one assertion. Static checks do not establish browser usability or hosted operation.

9.2 Workspace regression and local API checks

Table 12b. Added workspace unit tests in tests/workspace.test.mjs

ID	Concrete input and expected result	Result
UT-08	Edit a copied template; original input and persisted baseline remain unchanged.	Pass
UT-09	Create in storage A; reopen A retains it, storage B does not contain it.	Pass
UT-10	Change host country; previous responses and assessment timestamp clear.	Pass
UT-11	Save revision n, then retry n; reject and preserve the newer name.	Pass
UT-12	Save Documented with a five-character note; reject without changing storage.	Pass
UT-13	Reset storage A; retain B and reject stale pre-reset revisions.	Pass
UT-14	Simulate a storage quota error; no successful save or persisted change.	Pass
UT-15	Read an unsupported workspace version; reject without silently replacing it.	Pass

Table 13. HTTP checks through the local Worker compatible runtime

ID	Request and expected result	Observed
API-01	GET templates after local migration; four fictional examples returned.	200; Pass
API-02	Anonymous POST to projects; creation denied.	403; Pass
API-03	Direct anonymous PUT to a known template; mutation denied.	403; Pass
API-04	GET after mutation attempts; all template content equals the earlier snapshot.	Unchanged; Pass

Run `npm run test:api` to repeat migration and these HTTP checks. The script is now included in the source and CI configuration. The prior v1.0.0 SEC-01 defect accepted anonymous shared writes; API-02–04 verify that the revised server rejects them. Workspace tests use independent Storage-compatible test doubles, not two real browsers. Full hosted browser isolation still requires observation.

Evidence files: `quality_check_20260925.log`, `build_verification_20260925.log`, `deployment_dry_run_20260925.log` and `api_verification_20260925.log`. The accompanying source contains the executable tests. Docker, hosted performance, keyboard/zoom acceptance, maintenance timing and rollback were not executed in this review.

10 Requirements traceability and evaluation

The following mapping identifies what was demonstrated and what remains to be checked. A source-level implementation or local unit result is not equivalent to end-user or production acceptance. The browser workspace change is reflected in requirements, design and evidence.

Table 14. Functional requirements and evidence

Requirement	Implementation and available evidence	Remaining observation
FR-01–03	Portfolio and profile; schema tests, UT-09–11 and API-01.	Browser create, select and edit walkthrough.
FR-04–05	Local save and evidence checks; UT-09, UT-12 and API isolation checks.	Browser save/reopen and error presentation.
FR-06–08	Conditional rubric, score, status and actions; UT-01–03 and §8 worked example.	Visual interpretation and action ordering.
FR-09–10	Text report and framework content; UT-04 plus source inspection.	Browser download and reference links.

Table 15. Non-functional requirements and evidence limits

Requirement	Assessment as of 25 September
NFR-01	Input schemas and selected invalid inputs tested; exhaustive boundaries unverified.
NFR-02	Revision-conflict unit checks pass; cross-tab Web Locks behaviour requires browser observation.
NFR-03	Rule source, worked arithmetic and domain tests support explainability; no independent user study.
NFR-04	Direct shared writes rejected and separate test stores isolated; hosted two-profile observation open.
NFR-05	Local migration and checks executed; clean install duration and Docker execution unverified.
NFR-06	60-minute maintenance scenario not timed; target remains unevaluated.
NFR-07	Keyboard, 200% zoom and 360 px observations not executed in this review.
NFR-08	Hosted p95 target not measured; no performance pass claimed.
NFR-09	Fictional examples and absence of identity/payment fields confirmed by source inspection.
NFR-10	Rollback procedure documented; 60-minute recovery target not rehearsed.

11 Installation, operation and release

11.1 Local operation and reviewer walkthrough

Prerequisites are the complete extracted source tree and a supported Node/npm environment. The source specifies Node ≥ 22.13 ; CI is configured for Node 22. The current review used Node 24.19.0. Run commands from the application root, where `package.json` is located. Account creation is not required for local D1 emulation.

```
npm ci
npm run db:migrate:local
npm run check
npm run dev
```

Open the local URL printed by the development server, normally `http://localhost:5173`. Select a fictional example; inspect its profile and applicable criteria; change a rating and evidence note; save and reopen; inspect gaps and download the text report. For a controlled validation check, select Documented with a short note and confirm rejection. Check the notice explaining local storage, and use Reset my demo only after exporting any work to retain.

Docker Compose procedure

The supplied Compose file starts the development server and keeps dependencies and local D1 state in named volumes. It does not apply the migration automatically. The following sequence adds the necessary database-initialisation step (Docker, n.d.):

```
docker compose build
docker compose run --rm app npm run db:migrate:local
docker compose up
```

Open `http://localhost:5173`. Use `docker compose down` to stop the environment while preserving named volumes. Removing volumes also removes local database state. Docker execution was not available in this review, so this procedure is a corrected rehearsal instruction, not a completed test result.

Common operational failures

Symptom	Specific response
Projects fail to load or API returns 503.	Check that the local or remote migration was applied to the intended D1 database and that the DB binding matches configuration.
A local save reports a stale revision.	Reload the latest project before reapplying the edit; do not overwrite newer content blindly.
Documented evidence is rejected.	Supply a meaningful fictional reference of at least ten characters after trimming, within the evidence length limit.
Port 5173 is already in use.	Select a free local port using the development command's port option; open the URL the server prints.

11.2 GitHub, Cloudflare and final source freeze

GitHub stores source and runs workflows; uploading a ZIP alone does not provision the Worker or its database. The inspected export includes Cloudflare deployment material, but no successful public deployment is evidenced. The sequence below is the remaining release procedure after the local write-isolation checks (Cloudflare, n.d.; GitHub, n.d.-a).

Step	Required action and exit evidence
1. Prepare source	Extract the archive and upload the project files with their directory structure, including workflow/configuration files. Keep installed dependencies, local database state and tokens out of Git.
2. Provision Cloudflare	Create or select the account and D1 database; obtain a token with the Worker deployment and D1 permissions needed by this project, restricted to the intended account.
3. Set deployment values	Supply CLOUDFLARE_ACCOUNT_ID, CLOUDFLARE_API_TOKEN and CLOUDFLARE_D1_DATABASE_ID using the repository deployment settings expected by the supplied workflow.
4. Check configuration	Confirm the Worker name, database binding and database identifier. The supplied configure-deployment script fills configuration from environment values; do not commit the API token.
5. Deploy and verify	Run the supplied deploy-cloudflare workflow; apply the remote migration and publish the Worker. Open the resulting HTTPS URL and execute the core workflow plus isolation checks.
6. Freeze the release	Record the actual public repository URL, application URL, successful workflow and immutable source commit. Tag the accepted release and create the implementation ZIP from that same commit.

The account ID identifies the account, the D1 database ID identifies the database, and the API token authorises deployment operations. They are not a reviewer's application login. Only the token is an authentication secret; this workflow may store all three values as GitHub secrets for consistent configuration. The academic demonstration is intended to open without an application account.

Source equivalence and submission integrity

Compare the relative file names and file-content hashes in the final implementation ZIP with an archive of the recorded Git commit, ignoring only the archive's enclosing directory. Do not compare only the ZIP byte hashes: regenerated source archives may use different compression while containing identical files (GitHub, n.d.-b). Include required source, assets, migrations, configuration and documentation; exclude secrets and generated local state.

The university requires GitHub and the submitted implementation source to remain identical and unchanged after final submission (IU, n.d., pp. 4–5). Complete repairs, deployment verification and packaging before the freeze; preserve the assessed release thereafter.

12 Technical debt and critical reflection

Table 16. Technical debt and release decisions

ID / issue	Decision and closure evidence	Priority
TD-01 Real-data tenancy	Accounts and authenticated ownership are outside the fictional-data scope. Reassess only if a real-data service is proposed; browser profiles are not authenticated user accounts.	Accepted scope limit
TD-02 Rubric history	Freeze v1.0 for the assessed release. Before later rule changes, persist or otherwise identify the rubric applied to saved assessments and explain comparability.	Documented debt
TD-03 Application PDF	Retain the implemented text report. Application PDF styling is optional; the university's separate PDF abstract remains mandatory.	Deferred
TD-04 Regression breadth	Workspace and local HTTP checks are integrated and pass. Full browser acceptance remains to be recorded.	Partly closed
TD-05 Quality measurements	Complete keyboard, zoom, viewport and timing observations; record maintenance and rollback scenarios or explicitly report unmet targets.	Acceptance evidence
TD-06 Assessment history	The latest response set is stored without an audit/history table. This is acceptable for fictional demonstration data.	Accepted scope limit
TD-07 Deployment proof	Execute clean Compose and Cloudflare rehearsals; record actual URLs, release commit and rollback result.	Release blocker
TD-08 Anonymous shared writes	Code fixed in v1.0.1. API-02–04 reject server mutations; isolated-store regression tests pass. Confirm on the public deployment.	Locally closed; hosted check open

The central engineering contribution is the connection between profile-dependent applicability, weighted evidence assessment, critical gates and actionable output. Separating the domain functions made the distinction between completeness and readiness directly testable. Local API checks added evidence that source inspection alone could not provide, including a reproduced security defect.

The process also exposed shortcomings. The initial specification overcommitted to accounts, administration and a different backend, while later documentation sometimes described planned controls as completed behaviour. The revised approach ties each claim to a version and evidence source. A single maintained report reduces contradictions, but historical submissions are retained as the development record.

Evaluation remains limited by a solo author, self-reported inputs, a small synthetic dataset and no independent user study. The proposed customer value is credible as a design objective but is not an empirically established outcome. Further academic work could validate the rubric with domain experts and compare task completion against a spreadsheet baseline; no such study is claimed here.

13 Final submission assessment and packaging

The course brief, rather than the Phase 1 proposal, controls final packaging. The required two-page making-of abstract is separate from the consolidated specification (IU, n.d., pp. 4–5, 10, 12–13). The report, abstract and revised source are prepared. Public URLs, final source equivalence and university upload remain student-controlled actions due by the 26 September deadline.

Table 17. Required deliverables and current state

Required element	Current assessment / final action
Working original cloud application	Revised source and local checks are evidenced. Public URL and hosted acceptance remain open.
Public GitHub and identical code ZIP	Version 1.0.1 code ZIP is supplied. Publish its extracted contents; record public URL and final commit; regenerate ZIP if any source changes.
Docker Compose and installation files	Present in export; migration-first instructions corrected in §11.1. Clean Docker rehearsal remains open.
Project, requirements and architecture documentation	Consolidated in this report with page numbers, separate terminology lists and revised figures. Design and evidence are aligned with prepared v1.0.1.
Two-page making-of abstract PDF	Provided separately as Wabo-Yannick_Aurelien_9218017_PSE_P3_S.pdf; retain the stated distinction between local verification and pending hosted release.
Links text file	Populate with real repository and application URLs and reviewer login only if needed. Do not substitute Cloudflare deployment credentials.
Earlier phase submissions and final archive	Phase 1 PDF, a Phase 2 document uploaded on 17 September and the later revision are included. Confirm the exact submitted Phase 2 file.
University submission formalities	Complete the required electronic affidavit and upload through the prescribed course workflow.

Required archive organisation

Use the root Wabo-Yannick_Aurelien_9218017_PSE with folders 01-Project-management, 02-Requirements, 03-Architecture-documentation and 04-Implementation. The code archive is Wabo-Yannick_Aurelien_9218017_PSE_Submission_Code.zip. Keep one authoritative consolidated report; use an index or section extracts to connect its content to the prescribed folders rather than maintaining competing editable specifications. The package includes section extracts generated from this canonical report and labelled as extracts; edits should be made only in the canonical source.

Assessment against the marking dimensions

Problem solving (10%) is supported by the bounded preparation question. Methodology (20%) is strengthened by explicit feedback handling and observable work packages. Implementation/documentation (40%) has the greatest remaining exposure: hosted operation, browser acceptance and release equivalence still need proof. Creativity/correctness (20%) is supported by the explainable rubric but limited by unvalidated weights. Formal requirements (10%) are addressed through consistent styles, pagination and required artifacts. These are evidence-based judgements, not a predicted grade (IU, n.d., p. 9).

14 List of abbreviations

Abbreviation	Meaning
ADR	Architecture decision record; replaces the earlier D-01–D-05 identifier family.
API	Application programming interface; API-01–04 also identify executed HTTP checks
CI	Continuous integration
FR	Functional requirement
HTTP / HTTPS	Hypertext Transfer Protocol / HTTP protected by transport encryption
ICVCM	Integrity Council for the Voluntary Carbon Market
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
MRV	Monitoring, reporting and verification
NDC	Nationally determined contribution
NFR	Non-functional requirement
PACM	Paris Agreement Crediting Mechanism
QA	Quality assurance
R	Project-risk identifier
SEC	Security-check identifier
SQL	Structured Query Language
TD	Technical-debt identifier
UI	User interface
UML	Unified Modeling Language
UNFCCC	United Nations Framework Convention on Climate Change
UT	Unit-test identifier
WCAG	Web Content Accessibility Guidelines
WP	Work-package identifier

Identifiers such as FR-01, NFR-06, R-01 and ADR-05 combine the category above with a sequential number; they are cross-reference labels, not requirement priorities. Cloudflare D1 is a product name rather than an abbreviation expanded in the glossary.

15 Glossary

Term	Definition within this project
Additionality	A screening topic concerning whether mitigation depends on the relevant carbon-credit incentive, as addressed by the applicable method or programme.
Applicable criterion	A rubric item included for the project's selected sector, pathway and intended use.
Assessment completeness	The proportion of applicable criteria with a selected rating, independently of the readiness score.
Authorisation	A formal decision by a relevant Party under applicable arrangements; an application score does not provide it.
Corresponding adjustment	A Party-level accounting concept screened as an evidence and responsibility topic where applicable; the software does not perform official accounting.
Critical criterion	An applicable item that must receive full documented credit before the application can display Evidence ready.
Evidence note	Text supplied by a user identifying or describing supporting material; its content is not independently verified.
Fictional demonstration	An illustrative project and evidence set created for software demonstration rather than a verified real mitigation activity.
Mitigation contribution	The project's selected contribution-use scenario for which the application does not seek international-use authorisation in its rubric.
Optimistic concurrency	An update approach that detects whether a saved record changed after it was read and rejects a conflicting stale save.
Readiness score	The rounded weighted measure of declared evidence maturity across applicable rubric criteria.
Release freeze	The point at which the assessed source version and submitted files are fixed after verification.
Revision	The increasing integer used by the implementation to identify a newer saved project state.
Rubric	The versioned set of criteria, weights, applicability rules, rating values and status conditions used for screening.
Technical debt	A known implementation or verification limitation with an explicit decision, priority and closure condition.
Visitor sandbox	An isolated space for fictional user experiments; the selected final design uses browser-local copies and reset without altering shared examples.

This glossary defines concepts used in the report. Acronym expansions and identifier families are intentionally confined to the preceding, separately alphabetised list.

16 References and supporting evidence

Beck, K., et al. (2001). Principles behind the Agile Manifesto. <https://agilemanifesto.org/principles.html>

Cloudflare. (n.d.). Deploy to Cloudflare Workers with GitHub Actions. <https://developers.cloudflare.com/workers/ci-cd/external-cicd/github-actions/>

Course assessor. (n.d.). Feedback on Phase 2 and follow-up guidance on planning, quality targets and notation. Unpublished course communications supplied by the student.

Docker. (n.d.). docker compose run; Compose quickstart. <https://docs.docker.com/reference/cli/docker/compose/run/> and <https://docs.docker.com/compose/gettingstarted/>

GitHub. (n.d.-a). Using secrets in GitHub Actions. <https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets>

GitHub. (n.d.-b). Downloading source code archives. <https://docs.github.com/en/repositories/working-with-files/using-files/downloading-source-code-archives>

IU International University of Applied Sciences. (n.d.). Project: Software Engineering (DLMCSPSE01): Assignments portfolio. Supplied course brief, 13 pages; Task 1, finalisation requirements and formal instructions.

International Organization for Standardization. (2018). ISO/IEC/IEEE 29148:2018: Systems and software engineering—Life cycle processes—Requirements engineering. Public catalogue description. <https://www.iso.org/standard/72089.html>

MDN contributors. (n.d.). Web Locks API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Locks_API

Object Management Group. (2017). Unified Modeling Language, version 2.5.1. <https://www.omg.org/spec/UML/2.5.1/>

United Nations Framework Convention on Climate Change. (n.d.). Article 6.4 rules and regulations. <https://unfccc.int/process-and-meetings/bodies/constituted-bodies/article-64-supervisory-body/rules-and-regulations>

Wabo, Y. A. (2026a, September 20). ClimateReady Africa: Revised Phase 1 specification and Phase 2 development/reflection documentation (Version 2.1). Unpublished portfolio document.

Wabo, Y. A. (2026b, September 25). ClimateReady Africa (Version 1.0.1) [Prepared source code]. Revised from the supplied v1.0.0 export; file manifest accompanies the submission package.

World Wide Web Consortium. (2024). Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation. <https://www.w3.org/TR/WCAG22/>

Web documentation was accessed on 25 September 2026. Table 9 provides primary documentation links for the selected technologies. The ISO reference is limited to the public catalogue description; it does not imply access to, or compliance with, the full standard.

Supporting evidence inventory

The code documentation includes current quality, build, deployment dry-run and local API logs. The API log includes migration output; executable tests are retained in the source. Earlier source material comprises the university brief, Phase 1 PDF, available Phase 2 v2.1 revision and the v1.0.0 source export. The review adds no claim that a production deployment, public repository or final frozen submission has already been completed.